TECHNICAL ARCHITECTURE & SYSTEM SPECIFICATION

Scribe AI: Full-Stack Gemini Assistant

Comprehensive Technical Manual on RAG, Function Calling, Streaming SSE, Voice, Authentication, and DevOps Architecture

Author: Sahil Pathan

Engine: Google Gemini 3.5 & Vector Store

Stack: Node.js, Express, Docker, CI/CD

Date: October 2026

01 System Overview & Architecture Flow

Scribe AI is an enterprise-grade multimodal assistant built using Node.js (ES Modules) and Google's latest generative models. The application is engineered around an asynchronous, non-blocking pipeline combining document intelligence (RAG), bidirectional voice streaming, dynamic tool dispatching, persistent state memory, and hardened containerized infrastructure.

Layer	Key Technologies	Core Responsibility
Client Interface	HTML5, Vanilla CSS (Glassmorphism), Web Speech API, EventSource SSE	Microphone capture, audio playback visualizer, token streaming display, document upload drag-and-drop.
API Gateway	Express 4, Multer, Rate Limiter, Helmet-style CSP, JWT Middleware	Input validation, user/guest scoping, route authorization, file upload limits, and SSE protocol termination.
Intelligence Core	`@google/generative-ai` (Gemini 3.5 Flash Lite)	Streaming reasoning, candidate preservation, thought-signature tracking, and tool execution orchestration.
Vector Memory (RAG)	`PDFParse` v2, `gemini-embedding-001`, VectorStore	Text chunking with sliding boundary overlap, 3072-dim embeddings, and cosine similarity ranking.
Storage & Persistence	Atomic JSON Stores (`db.json`, `embeddings.json`)	Multi-user scoping, isolated corruption recovery, chat history, and sanitized sharing snapshots.
Infrastructure & CI/CD	Docker Multi-Stage Alpine, Compose, GitHub Actions	Containerized isolated execution, volume persistence, syntax/build automation, and zero-downtime deployment.

02 Feature-by-Feature Deep Dive

1. RAG (Retrieval-Augmented Generation — Chat with Documents)

What it does: Allows users to upload PDF, text, or markdown files and ask natural language questions grounded exclusively or primarily in the contents of those documents, complete with cited source references.

How it works under the hood:

- **Parsing (`lib/rag.js`):** PDF streams are parsed via `PDFParse` v2. Binary data is extracted into raw unicode text while handling complex page headers and breaks.
- **Sliding-Window Chunking (`lib/vectorStore.js`):** Text is split into bounded segments of 750 characters with a 120-character overlapping sliding window. The algorithm evaluates paragraph breaks (`\n\n`), line breaks (`\n`), and sentence boundaries (`. `) to avoid slicing thoughts in half. Loop step-increments are guarded to guarantee termination.
- **Chunk Bounds & Quota Protection:** Document uploads are enforced with a `MAX_CHUNKS_PER_DOC = 100` limit to avoid burning API quota or triggering HTTP timeout cascades.
- **Vector Embeddings (`gemini-embedding-001`):** Every extracted chunk is transformed into a dense 3,072-dimensional semantic vector using Gemini's embedding model.
- **Cosine Similarity Retrieval:** When a user asks a query, the query itself is embedded. The vector store computes the dot product of normalized vectors:

  ```
  cosineSimilarity(a, b) = dotProduct(a, b) / (norm(a) * norm(b))
  ```

  Chunks matching above a strict similarity threshold (`> 0.35`) are gathered, ranked, and the top-4 most relevant excerpts are formatted into grounded system context before querying Gemini.

2. Persistent Context Memory & Auto-Summarizer

What it does: Remembers user conversations across browser refreshes and sessions. Long conversations that would normally blow past token limits or become expensive are automatically compressed into executive summaries.

How it works under the hood:

- Chats and messages are persisted in `data/db.json` with per-user isolation.
- When a conversation exceeds 8 message turns (`CONTEXT_WINDOW_LIMIT = 8`), the engine triggers an automatic background summarizer via Gemini (`lib/gemini.js`).
- Gemini produces a concise running summary of older exchanges. Scribe saves this in `chat.summary` .
- On subsequent requests, the prompt is structured as:

  ```
  [System Persona & Instructions]
  [Running Summary of Older Discussion: chat.summary]
  [Retrieved Grounded Document Excerpts (if any)]
  [Recent Verbatim Messages (last 8 turns)]
  [Current User Message]
  ```

  This maintains infinite conversational recall with a compact, cost-efficient token footprint.

3. Real-Time Token-by-Token Streaming (Server-Sent Events)

What it does: Rather than waiting 5–10 seconds for the entire AI answer to generate, words appear on screen immediately as they are generated by the model.

How it works under the hood:

- The server initiates an HTTP Server-Sent Events (SSE) stream via headers:

```
Content-Type: text/event-stream
Cache-Control: no-cache, no-transform
Connection: keep-alive
```

- The server calls `model.generateContentStream({ contents })`. As each token or candidate chunk is emitted by Gemini, it is immediately written to the HTTP response socket as:

```
data: {"text": "token text..."}\n\n
```

- On the frontend, the fetch response body is consumed via `ReadableStreamDefaultReader`. A live markdown transformer parses headers, bullet lists, blockquotes, and code wrappers in real time.
- When Gemini completes, the server writes `data: [DONE]\n\n` and cleanly closes the response.

03 Function Calling & Multimodal Features

4. Autonomous Function Calling (Live Tools)

What it does: Allows Gemini to autonomously recognize when it needs external information or computation and execute real tools on the server, returning live data to the conversation.

How it works under the hood:

- **Tool Declarations (`lib/tools.js`):** Defined using the Gemini Function Declaration OpenAPI schema:
  - `get_weather`: Queries Open-Meteo Geocoding and Weather REST APIs for real-time temperature, wind, and conditions.
  - `search_web`: Queries DuckDuckGo Instant Answers & Wikipedia APIs for live facts, current events, and references.
  - `send_email`: Simulates email dispatch with validated recipient, subject, and message content.
  - `query_database`: Queries user-scoped metrics (number of active chats, uploaded documents, stored vectors).
  - `calculate`: Performs safe arithmetic and mathematical evaluations without arbitrary code execution.
- **Thought-Signature & Candidate Preservation (`lib/gemini.js`):** Gemini 3.5 requires preserving model candidate parts (including `thoughtSignature` and function call `id`). Scribe maintains the exact candidate parts and responds with `role: "user"` and `parts: [{ functionResponse: { name, response } }]` to finalize the stream.

5. Voice Input & Voice Output (Speech Pipeline)

What it does: Users can speak into their microphone to dictate prompts, and toggle an audio speaker button to have Scribe speak replies aloud with real-time waveform visualizers.

How it works under the hood:

- **Speech-to-Text (STT):** Built with the browser's native `webkitSpeechRecognition` / `SpeechRecognition`. Listens continuously, updates the composer input, and animates a dynamic purple audio visualizer pill during dictation.
- **Text-to-Speech (TTS):** Utilizes `window.speechSynthesis`. Text is stripped of markdown syntax, code snippets, and URL strings before synthesis so the speech audio sounds natural. Includes an auto-speak toggle saved in `localStorage`.

6. Authentication & Multi-User Data Scoping

What it does: Provides secure registration and login, while also supporting an anonymous guest mode that never mixes data between different visitors.

How it works under the hood:

- **Password Security:** Passwords hashed with `bcryptjs` (10 salt rounds). Login timing attack protection is enforced via a constant-time dummy hash verification for nonexistent accounts.
- **JWT Tokens:** Signed with `HS256` and a 32+ character server secret. Tokens expire in 7 days and are checked exclusively via `Authorization: Bearer <token>` headers.
- **Guest Isolation:** Every visitor without an account is issued a unique `x-guest-id` stored in their browser. All chats and documents are strictly scoped to `guest_<id>`, preventing users behind corporate NATs or

proxies from viewing each other's data.

7. Export & Cryptographic Social Sharing

What it does: Export discussions as formatted PDF, Markdown, or JSON files. Generate unguessable web snapshot links that can be shared with colleagues or friends.

How it works under the hood:

- **Snapshot Sanitization (`server.js`):** Shared chats generate a 16-byte base64url random ID (`sh_...`). The snapshot captures only `role`, `text`, and `createdAt`, stripping confidential document previews or internal tool payloads.
- **Standalone Viewer (`public/share.html`):** A dedicated read-only viewer displays the shared dialogue with zero inline scripts, adhering strictly to Content Security Policy standards.

04 DevOps, Containerization & Security Engineering

8. Production Docker Architecture

The included `Dockerfile` implements a multi-stage Alpine build designed for minimal image size and maximum runtime security:

- **Stage 1 (Builder):** Installs dependencies via `npm ci --only=production` and clears the npm cache.
- **Stage 2 (Runner):** Copies only production node modules and app code into a lightweight `node:20-alpine` image.
- **Non-Root Execution:** Drops privileges to user `node` (UID 1000). The data directory `/app/data` is pre-created and chowned to ensure unprivileged persistence.
- **Healthcheck:** Configured with `wget -qO- http://localhost:3000/api/health` checking liveness every 30 seconds.

9. Docker Compose Orchestration (`docker-compose.yml`)

Provides one-command initialization for production deployment. Mounts the named volume `scribe-data` to `/app/data`, guaranteeing that all registered users, chat histories, uploaded documents, and vector embeddings survive host reboots or container upgrades.

10. Automated CI/CD Pipeline (`.github/workflows/ci-cd.yml`)

Automates continuous integration and continuous deployment:

1. **Syntax & Unit Validation:** Executes `node --check` on all backend and frontend JS files. Runs vector chunking mathematical verification.
2. **Docker Build Verification:** Compiles the container image to ensure build reproducibility.
3. **Continuous Deployment:** When pushed to the `master` branch, triggers a secure webhook to Render or Railway to deploy the new container instantly with zero downtime.

11. Rate Limiting & Defense in Depth

- **Rate Limiting:** In-memory sliding window limiter restricting requests to 60 per minute per IP, protecting the Gemini API key from denial-of-wallet attacks.
- **Upload Limits:** Multer limits uploads to 10MB and restricts MIME types strictly to PDF, TXT, and Markdown, returning JSON `413` and `400` error envelopes.
- **Security Headers:** Strict Content-Security-Policy (CSP), `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, and `Referrer-Policy: strict-origin-when-cross-origin`.
